

Кроме операторов, направленных на решение специфических задач управления данными (**INSERT**, **UPDATE**, **SELECT** и т.д.), **Transact SQL** поддерживает основные операторы управления ходом программы.

Поддерживаются условный оператор **if ... else**, оператор организации цикла **while**, оператор безусловного перехода **goto**.

Кроме того, **Transact SQL** поддерживает написание подпрограмм (сохраненных процедур), которые могут быть вызваны как из клиентского приложения, так и из другой процедуры.

Сохраненные процедуры допускают рекурсивные вызовы, но максимальный уровень вложенности вызовов ограничен числом **16**. Написание сохраненных процедур будет рассмотрено далее.

Внутри кода на **Transact SQL** поддерживается объявление и использование переменных.

Также поддерживаются «общепринятые» инструкции **break**, **continue**, **return**.

Специфичными для **Transact SQL** являются выражения **waitfor**, **print** и **raiserror**.

1. Операторные скобки.

Так же, как и в подавляющем большинстве универсальных языков высокого уровня, в **Transact SQL** имеется возможность выделения части кода как единого блока. Для этого используются ключевые слова **begin** и **end** (аналогично языку Паскаль).

В большинстве случаев комбинация **begin ... end** используется совместно с условными операторами **if ... else** или оператором цикла **while** в том случае, когда ветка, выполняемая по условию, или тело цикла состоит более чем из одного SQL-выражения:

```
begin  
    /* объединяемые операторы */  
end
```

2. Операторы условного перехода и цикла.

Синтаксис условного оператора **if ... else** и оператора организации цикла **while** аналогичен синтаксису этих операторов в подавляющем большинстве современных языков высокого уровня:

```
if логическое_выражение оператор  
[else [if логическое_выражение] оператор]
```

```
while логическое_выражение оператор
```

В качестве логического выражения может выступать любое выражение, принимающее значение «истина» или «ложь». Могут использоваться логические операторы **and**, **or**, **not** и т.д. В логическом выражении могут участвовать запросы к таблицам, выражение **exists**. Оператор, выполнением которого управляет конструкция **if ... else** или **while**, может быть как простым (одиночным), так и составным. В последнем случае для его описания используется конструкция **begin ... end**.

Примеры.

```
if( exists( select 1 from sysobjects where name = 'MyTable' ) )  
    drop table MyTable          -- Управляется оператором if  
create table MyTable( id int ) -- Выполняется в любом случае
```

```
if( exists( select 1 from sysobjects where name = 'MyTable' ) )  
    begin                      -- Оператором if управляется весь блок  
        drop table MyTable  
        create table MyTable( id int )  
    end
```

```
declare @i int,  
        @s varchar(10)  
  
select @i = 0, @s = ''  
while( @i < 10 )  
    begin  
        select @s = @s + @i  
        select @i = @i + 1  
    end
```

3. Операторы break, continue и return.

Операторы **break** и **continue** служат для «принудительного вмешательства» в работу цикла **while**. Оператор **break** прерывает выполнение всего цикла, а **continue** прерывает выполнение текущей итерации и инициирует переход к следующей.

Оператор **return** служит для прерывания работы сохраненной процедуры и возврата статусного значения. В **Transact SQL** сохраненные процедуры могут возвращать только целочисленные статусные значения (будет рассмотрено позднее).

```
declare @i int,  
        @s varchar(10)  
  
select @i = 0, @s = ''  
while( 1 = 1 ) -- Всегда "истина"  
begin  
    if( @i >= 10 ) break          -- Если @i >= 10, прерываем цикл  
    select @i = @i + 1  
    if( @i % 2 = 0 ) continue -- Будем обрабатывать только нечетные @i  
    select @s = @s + @i  
end
```

4. Глобальные переменные.

Кроме рассмотренных ранее локальных переменных, имена которых начинаются с символа @, Transact SQL поддерживает глобальные переменные.

Глобальные переменные не требуют определения. Их имена начинаются с символов @@. Их область действия распространяется либо на сервер в целом, либо на данную сессию.

Основные глобальные переменные уровня сессии:

@@error	Код завершения последней операции, выполненной в рамках данной сессии. Переинициализируется после выполнения любой операции, даже инструкции if .
@@identity	Последнее значение поля IDENTITY , сгенерированное для данной сессии.
@@trancount	Уровень вложенность транзакции для данной сессии. Каждый begin transaction увеличивает значение этой переменной, каждый commit или rollback transaction - уменьшает.
@@transtate	Статус текущей транзакции после выполнения последней операции. В отличие от @@error , изменяется только при возникновении ошибок.
@@sqlstatus	Статус выполнения последней операции fetch (будет рассмотрено позднее).
@@nestlevel	Уровень вложенности вызовов сохраненных процедур или триггеров. Максимальное значение – 16 , после его превышения транзакция прерывается

Кроме этого, существуют переменные, позволяющие получить информацию о конфигурации и настройках сервера, отслеживать текущую активность и прочее. Эти переменные подробно описаны в документации по ASE.

В качестве примера можно привести переменную **@@version**, которая содержит полную версию ASE, на котором выполняется процесс.

5. Инструкция **waitfor**.

Инструкция **waitfor** позволяет задерживать выполнение процесса на определенное время или до наступления какого-либо события. Общий синтаксис **waitfor** таков:

```
waitfor { delay "time" | time "time"  
          | errorexit  
          | processexit  
          | mirrorexit }
```

Параметр **delay** “time” позволяет задержать выполнение текущего процесса на заданный интервал времени. Время указывается в формате “ЧЧ:ММ:СС”.

Параметр **time** “time” позволяет задержать выполнение текущего процесса до наступления указанного момента времени. Время указывается в формате “ЧЧ:ММ:СС”.

Параметр **errorexit** задерживает выполнение текущего процесса до тех пор, пока какой-либо другой процесс не завершится в связи с возникновением ошибочной ситуации.

Параметр **processexit** задерживает выполнение текущего процесса до тех пор, пока какой-либо другой процесс не завершится в по любой причине.

Параметр **mirrorexit** задерживает выполнение текущего процесса до тех пор, пока не возникнет ошибка чтения или записи на дублирующее устройство (mirror device).

Параметры **delay** и **time** обычно используются для организации периодических процессов, выполняющих что-либо с заданным таймаутом или в определенное время.

Параметры **errorexit**, **processexit** и **mirrorexit** могут использоваться в процессах, осуществляющих мониторинг работы сервера.

Пример. Чистка старых записей каждый час в течение рабочего времени.

```
-- Вечный цикл, для завершения будем убивать процесс  
while( 1 = 1 )  
  begin  
    -- Стартуем в 8:00  
    waitfor time "8:00:00"  
  
    -- Работаем до 22:00  
    while( datepart( hour, getdate() ) < 22 )  
      begin  
        -- Удаляем записи "старше" 2 часов  
        delete from SomeTable  
          where RecordDateTime < dateadd( hour, -2, getdate() )  
  
        -- Задержка в 1 час  
        waitfor delay "1:00:00"  
      end  
    end  
end
```

6. Инструкция print.

Инструкция **print** используется для создания серверного сообщения. Это сообщение может быть отображено при помощи **isql**, **SQL Advantage** или другого клиентского приложения, исполняющего текущий процесс и поддерживающего вывод серверных сообщений.

Синтаксис **print**:

```
print { format_string | @local_variable | @@global_variable } [, arg_list]
```

Первый аргумент – строка формата, которая может быть строковой константой или переменной (максимальная длина – 255 символов).

Эта строка может содержать метки, вместо которых в сообщение будут подставлены значения последующих аргументов. Метка для первого (следующего сразу за строкой формата) аргумента имеет вид **%1!**, второго – **%2!** и т.д. Метки могут быть расположены в любом порядке, но не должно быть пропусков. Например, нельзя использовать метки **%1!** и **%3!**, но не включить в строку формата метку **%2!**.

После строки формата следует список аргументов, значения которых будут включаться в сообщение. Аргументами могут выступать константы или переменные. Использовать в качестве аргументов вызовы функций нельзя. Максимальное количество этих аргументов – **20**.

Длина результирующего сообщения ограничена **512** символами.

```
declare @cnt int,  
        @dt  datetime  
  
select @dt = getdate()  
select @cnt = count(*) from Clients  
  
print 'На момент %1! в таблице Clients %2! записей', @dt, @cnt  
  
/* или так, поменяв метки и аргументы местами:  
print 'На момент %2! в таблице Clients %1! записей', @cnt, @dt  
*/
```

7. Raiserror.

Инструкция **raiserror** позволяет генерировать ошибочную ситуацию с указанным номером и текстом сообщения. Синтаксис **raiserror**:

```
raiserror error_number [{format_string | @local_variable}] [, arg_list]
    [with errordata extended_value = extended_value [,
    extended_value = extended_value [...]]]
```

- Первый параметр – код ошибки. Этот код помещается в глобальную переменную **@@error**. Для пользовательских сообщений код должен быть больше **17000**.
- Второй параметр – строка формата. Правила ее построения аналогичны строке формата инструкции **print**. Если код ошибки находится в пределах **17000-19999** (этот диапазон используется системными сохраненными процедурами) и строка формата отсутствует или является пустой (“”), то сервер ищет соответствующий текст сообщения в таблице **master..sysmessages** по коду ошибки.
- Далее следуют аргументы, используемые для формирования сообщения (правила аналогичны инструкции **print**).
- При необходимости после ключевых слов **with errordata** можно указать расширенную информацию об ошибке (**extended_values**). Эти данные могут быть использованы приложением **Open Client**.

Пример.

```
-- Генерация сообщения об ошибке с передачей доп. параметров для
-- клиентского приложения
raiserror 25001 'Ошибка при создании платежа на сумму %1! со счета %2!',
    @amount, @accountFrom
with errordata amount = @amount,
    account_from = @accountFrom,
    account_to = @accountTo
```


8. Создание сообщений для print и raiserror.

В ASE существует возможность создания пользовательских сообщений с целью их дальнейшего использования в инструкциях **print**, **raiserror** или по своему усмотрению.

Сообщения хранятся в таблице **sysusermessages**. Обычно этот механизм используется для создания нескольких вариантов одних и тех же сообщений на разных языках.

Регистрация сообщения производится при помощи системной процедуры **sp_addmessage**, получение – **sp_getmessage**.

Пример:

```
set language us_english
go

-- Английский язык (по умолчанию)
sp_addmessage 25001,
    'There is already user named "%1!" for server "%2!"'
go

-- Немецкий язык
sp_addmessage 25001,
    'Utzername "%1!" existiert bereits auf dem Server "%2!"',
    'german'
go

-- Использование
declare @msg varchar(255)
sp_getmessage 25001, @msg output
print @msg, 'user', 'server'
go
```

9. Понятие курсора.

Курсоры позволяют обрабатывать записи ResultSet'a по одной. Также возможна модификация или удаление записей на «индивидуальной основе».

Курсор представляет собой символьное имя, ассоциированное с выражением **SELECT**.

Основными понятиями, связанными с курсором, являются:

- **ResultSet курсора** – набор данных, сформированный в результате выполнения связанного с курсором запроса.
- **Текущая позиция курсора** – указатель на строку ResultSet'a курсора.

Основные этапы работы с курсором:

- Объявление курсора
- Открытие курсора
- Выборка строки данных
- Обработка полученной строки
- Закрытие курсора
- Уничтожение курсора

10. Объявление и уничтожение курсора.

Объявление курсора производится при помощи ключевого слова `declare`:

```
declare имя_курсора cursor for {выражение_select}  
[for {read only | update}]
```

Для выражения **SELECT**, связанного с курсором, нет никаких ограничений (за исключением невозможности использования ключевых слов **INTO** и **COMPUTE**). Может использоваться группировка, агрегатные функции, объединения при помощи **UNION** или **UNION ALL**. Если выражение **SELECT** содержит раздел **ORDER BY**, курсор будет перебирать строки в указанном порядке.

Режим работы курсора по умолчанию – только чтение (**for read only**). При необходимости можно создать курсор, позволяющий модифицировать данные в исходных таблицах (**for update**), но такой режим возможен не всегда (например, он недоступен при использовании группировки).

В рамках данного пакета (batch) или сохраненной процедуры в каждый момент времени может быть объявлен только один курсор с одним и тем же именем (даже если этот курсор закрыт). Поэтому при необходимости повторного объявления курсора с тем же именем необходимо сначала уничтожить существующий.

Уничтожение курсора производится при помощи инструкции

```
deallocate cursor имя_курсора
```

11. Открытие и закрытие курсора.

После объявления перед использованием курсор должен быть открыт. Открытие курсора производится при помощи инструкции

```
open имя_курсора
```

Именно в момент открытия производится выполнение связанного с курсором запроса и формирование ResultSet'a курсора. Поэтому, если при объявлении курсора были использованы какие-либо переменные, их значение анализируется в момент выполнения операции **open**. Это позволяет использовать единожды объявленный курсор с разными параметрами, присваивая использованным при его создании переменным разные значения и переоткрывая его.

Закрывается курсор при помощи инструкции

```
close имя_курсора
```

После объявления курсор может открываться и закрываться любое число раз, главное соблюдать «парность» открытий и закрытий. Т.е., перед каждым открытием курсор должен быть закрыт.

12. Выборка данных из курсора.

Выборка данных из открытого курсора производится при помощи инструкции `fetch`:

```
fetch имя_курсора into список_переменных
```

Порядок следования и типы переменных в списке должны соответствовать порядку следования и типам полей связанного с курсором выражения **SELECT**. При выполнении операции **fetch** курсор позиционируется на очередную строку своего `ResultSet`'а и заполняет переданные переменные соответствующими значениями полей.

Результат выполнения операции **fetch** можно выяснить, проанализировав значение переменной **@@sqlstatus**:

- **0** – операция `fetch` выполнена успешно
- **1** – ошибка
- **2** – достигнута последняя строка `ResultSet`'а курсора

По умолчанию каждая операция **fetch** производит выборку одной строки. Но можно «проходить» по нескольким строкам, выполнив инструкцию

```
set cursor rows число for имя_курсора
```

Таким образом можно обрабатывать, например, каждую третью строку, поскольку после выполнения **fetch** курсор позиционируется на последнюю обработанную строку `ResultSet`'а.

13. Работа с курсором в рамках batch и в рамках сохраненной процедуры.

В данном разделе мы несколько "забежим вперед" и затронем тему сохраненных процедур, которая будет рассматриваться более подробно позднее.

Дело в том, что работа с курсором в рамках пакета (**batch**), или скрипта, запускаемого, например, из **isql** и из сохраненной процедуры имеет свои особенности.

При объявлении курсора непосредственно из пакета (**batch**) инструкция **declare cursor** должна быть единственной инструкцией в пакете. Это накладывает ограничения на использование переменных при объявлении курсора, поскольку переменные, объявленные в рамках одного пакета, в другом не видны.

Пример.

Допустим, стоит задача перебрать всех клиентов, для каждого из них - все счета и вывести эту информацию при помощи инструкции **print**. При помощи курсора, объявляемого в рамках **batch**, эту задачу можно решить так:

```
-- Создаем временную таблицу для хранения текущего ID клиента
create table #tmpClient( ClientID int )
go

-- Объявляем курсор для выборки клиентов
declare ClientsCur cursor for
select ClientID, FirstName, LastName
    from Clients
for read only
go

-- Объявляем курсор для выборки счетов текущего клиента
declare AccountsCur cursor for
select AccountNum, Currency
    from #tmpClient c, Accounts a
    where a.ClientID = c.ClientID
for read only
go

-- Переменные для хранения выбираемых значений
declare @clientId    int,
        @firstName   varchar(30),
        @lastName    varchar(70),
        @accountNum  char(14),
        @currency    char(3)

-- Начинаем выборку клиентов
open ClientsCur
```

```

-- Выбираем первую строку
fetch ClientsCur into @clientId, @firstName, @lastName

-- Обработку ведем до тех пор, пока @@sqlstatus = 0. Если нет ни одного
-- клиента, то после первого fetch @@sqlstatus примет ненулевое значение
while( @@sqlstatus = 0 )
begin
    -- Вычищаем #tmpClient и помещаем туда ID текущего клиента
    truncate table #tmpClient
    insert into #tmpClient( ClientID ) select @clientId

    -- Выводим данные клиента
    print 'Client [%1!] %2! %3!', @clientId, @firstName, @lastName

    -- Начинаем выборку счетов. Поскольку выборка идет по связке
    -- с #tmpClient, в выборку попадут только счета текущего клиента
    open AccoutnsCur

    -- Первый fetch
    fetch AccoutnsCur into @accountNum, @currency
    -- Снова ориентируемся по @@sqlstatus
    while( @@sqlstatus = 0 )
    begin
        -- Выводим данные по счету
        print '  Account %1! - %2!', @accountNum, @currency
        -- Выбираем следующий счет
        fetch AccoutnsCur into @accountNum, @currency
    end
    -- Когда счета закончатся, закрываем курсор AccoutnsCur
    close AccoutnsCur

    -- Выбираем следующего клиента
    fetch ClientsCur into @clientId, @firstName, @lastName
end
-- Когда клиенты закончатся, закрываем курсор ClientsCur
close ClientsCur
go

-- Чистим ресурсы
deallocate cursor AccountsCur
go
deallocate cursor ClientsCur
go
drop table #tmpClient
go

```

Та же задача, но в рамках сохраненной процедуры, решается несколько проще:

```
-- Подробности объявления процедуры пока опускаем
create procedure printClientaAndAccounts
as
begin
    declare @clientId    int,
            @firstName   varchar(30),
            @lastName    varchar(70),
            @accountNum  char(14),
            @currency    char(3)

    -- Объявляем курсор для выборки клиентов
    declare ClientsCur cursor for
    select ClientID, FirstName, LastName from Clients

    -- Объявляем курсор для выборки счетов текущего клиента.
    -- Курсор связываем с переменной @clientId, значение которой
    -- будет анализироваться при открытии данного курсора.
    declare AccountsCur cursor for
    select AccountNum, Currency from Accounts
    where ClientID = @clientId

    open ClientsCur
    fetch ClientsCur into @clientId, @firstName, @lastName
    while( @@sqlstatus = 0 )
    begin
        print 'Client [%1!] %2! %3!', @clientId, @firstName, @lastName

        -- Начинаем выборку счетов. Поскольку выборка идет по связке
        -- с @clientId, в выборку попадут только счета текущего клиента
        open AccoutnsCur
        fetch AccoutnsCur into @accountNum, @currency
        while( @@sqlstatus = 0 )
        begin
            print '  Account %1! - %2!', @accountNum, @currency
            fetch AccoutnsCur into @accountNum, @currency
        end
        close AccoutnsCur

        fetch ClientsCur into @clientId, @firstName, @lastName
    end
    close ClientsCur

    deallocate cursor AccountsCur
    deallocate cursor ClientsCur
    drop table #tmpClient
end
```


14. Модификация данных курсора.

Если курсор был объявлен **«for update»**, с его помощью можно модифицировать в базовых таблицах связанного оператора **SELECT** (также возможно удаление).

Все модификации производятся после выполнения операции **fetch**, когда курсор позиционируется на нужную строку.

Модификации выполняются при помощи обычных выражений **UPDATE** и **DELETE**. При этом условие **WHERE** описывается как

```
current of имя_курсора
```

В приведенном примере производится перенумерация строк таблицы:

```
declare @n int,  
        @i int  
  
declare numCursor cursor for  
select Num from MyTable  
for update  
  
open numCursor  
  
select @i = 1  
fetch numCursor into @n  
while( @@sqlstatus = 0 )  
begin  
    -- Изменяем строку, соответствующую текущей позиции курсора  
    update MyTable set Num = @i where current of numCursor  
  
    select @i = @i + 1  
    fetch numCursor into @n  
end  
  
close numCursor  
  
deallocate cursor numCursor
```